

COM2021 Web Development

Web Development Exercise - Assessment Report

Part 1: Design and Implementation

1.1 Architecture and State Management

The website is implemented as a LAMP stack application hosted on a Microsoft Azure virtual machine, with Apache serving PHP files and no relational database. User authentication state is managed through PHP `$_SESSION` variables, which confirm a registered visit for the lifetime of the browser session. In addition, the username and chosen avatar are written to HTTP cookies with a 7-day expiry at registration time, making the profile accessible across restarts and across all four pages without repeated session checks.

Multi-user leaderboard persistence uses a server-side JSON flat file (`leaderboard_store.json`). Each entry in the file is keyed by username (`$store['users'][$username]`), meaning that submitting a score either creates a new record or updates the existing one for that user. This ensures only one best-score record exists per username, avoiding bloat of the file. The file is read on every leaderboard page load, sorted by total best score descending, and rendered directly into the HTML table.

The four required pages - `index.php`, `registration.php`, `pairs.php` and `leaderboard.php` each contain their own navbar markup inline. The conditional logic determining whether to show the Leaderboard or Register link is resolved individually in each file using the same `isset($_COOKIE['username'])` check, keeping each page self-contained. The global stylesheet is `styles.css`, linked from each page directly.

1.2 Landing Page

The landing page (`index.php`) starts a PHP session and inspects whether a username cookie is set. If so, it displays a welcome message and a 'Click here to play' button hyperlinked to `pairs.php`. If no cookie is present, it instead renders a paragraph containing a 'Register now' hyperlink to `registration.php`. All branching is resolved server-side so the correct HTML is delivered on first load with no client-side flicker.

1.3 Registration and Avatar Selection

The registration form posts back to `registration.php`, where the username field is validated server-side using PHP `preg_match` with the following regular expression:

```
/[!"@#%^\^*\(\)\+=\{\}\[\];:'''<>?\'\/]/u
```

Any input containing one of these characters causes an error message to be rendered immediately beneath the input field, with the previously typed value repopulated via `htmlspecialchars($_POST['username'])`. This prevents both data loss and reflected XSS, since the username is later echoed into the navbar and the welcome message. On successful validation, `setcookie()` stores the username and avatar identifier with `path=/'` and an expiry of 7 days.

I chose the medium difficulty avatar selector implementation, which presents a set of six image assets (`avatar1.png` through `avatar6.png`) as clickable radio inputs. Each image is wrapped in a label element; a JavaScript change handler adds a highlight class to the selected label to give clear visual feedback. A default option is pre-checked so that form submission always produces a valid avatar cookie value even without an explicit user selection. The stored avatar identifier is read from the cookie on subsequent page loads and used to display the image in the navbar.

1.4 Pairs Game

For the game itself, I chose the complex implementation for the biggest challenge. The game board is a JavaScript-driven interface rendered inside a grey-background div with a 5px box shadow, as specified. A 'Start the game' button initiates play and is then hidden. Cards are laid out in a CSS grid that recalculates column count based on the number of cards in the current level.

Card images are drawn from a fixed pool of emoji characters defined in a JavaScript array. At the start of each level, the required number of emoji are sampled from this pool and duplicated to form matched groups (pairs, triples, or quads depending on the level). The full set is then shuffled using a Fisher-Yates algorithm before being rendered onto the board, ensuring a different layout on every run.

Level progression follows a structured difficulty curve: Level 1 uses matched pairs (2 cards); Level 2 introduces triple matching (3 cards); Level 3 and beyond use quad matching (4 cards) with an increased number of cards on the board. The game enforces a selection limit equal to the matchSize for the current level. Once the player has selected matchSize cards, an attempt is recorded regardless of outcome. This definition of 'attempt' - a completed selection of matchSize cards, whether successful or not - is what the scoring algorithm uses as its penalty input, ensuring the metric is fair and consistent.

On a successful match, the selected cards are locked face-up. On a mismatch, they flip back after a 600 ms delay, allowing the player to register which faces they saw before the board resets. A CSS 3D rotateY transform on each card element handles the flip animation, with a prefers-reduced-motion media query disabling the animation for accessibility.

1.5 Scoring, Timer and Gold Background

A timer begins when the game starts and records elapsed seconds continuously. Per-level points are calculated from two components: a base score that decays with the number of failed attempts, and a time bonus that rewards faster completion. Both levelScores[] (an array of points per level) and totalScore (the running sum) are maintained in the game state object throughout the session.

After each successful match, the accumulated score for the current level is compared against the stored best for that level. If the live score exceeds the previous best at any point during gameplay, the background colour of the game container transitions to gold (#FFD700) using a CSS transition, providing a clear real-time visual signal that the player is on track for a personal best. The background reverts to grey at the start of each new level.

On game completion, if the player is in a registered session, a panel appears showing the total score, per-level breakdown, and two buttons: 'Submit Score' and 'Play Again'. Clicking Submit triggers a fetch() POST to leaderboard.php with the score data as JSON. Clicking Play Again resets all score state to zero and restarts from Level 1.

1.6 Leaderboard

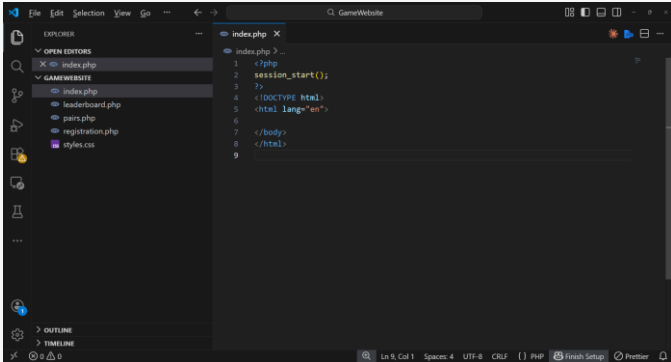
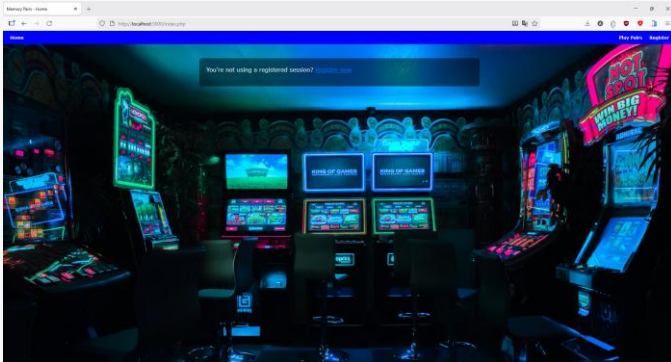
leaderboard.php handles both score submission (POST) and display (GET). On POST, the server reads leaderboard_store.json, updates the record for the submitting username (per-level bests and total best are updated individually only if the new value exceeds the stored one), and writes the file back. On GET, the file is read, entries are sorted by total best score descending, and the results are rendered in a table with blue header cells and border-spacing: 2px. Columns display the username, individual level best scores, and the overall best total.

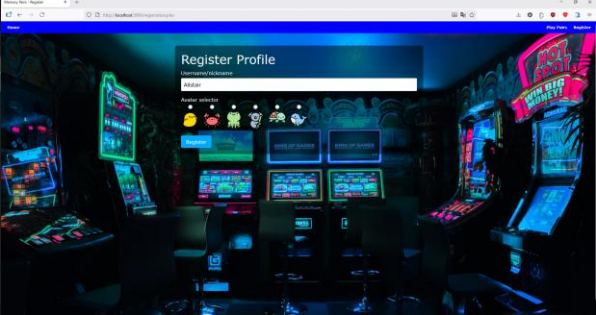
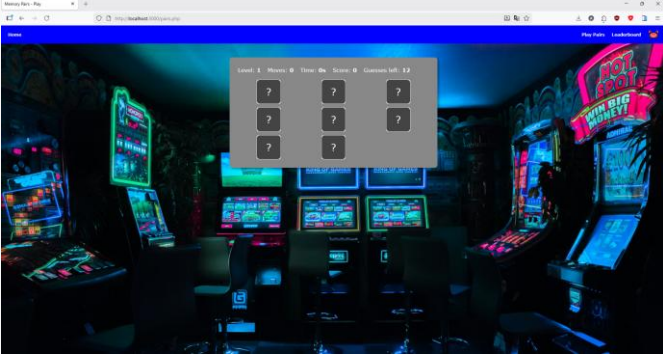
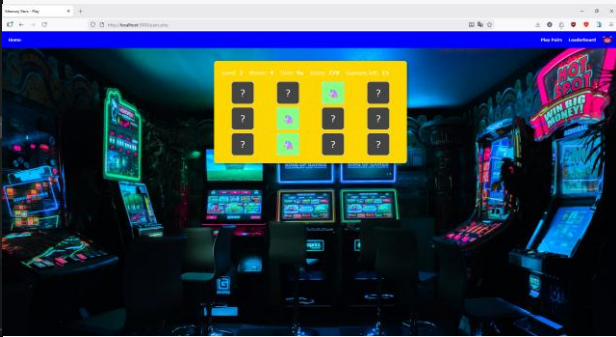
1.7 Known Design Issues and Limitations

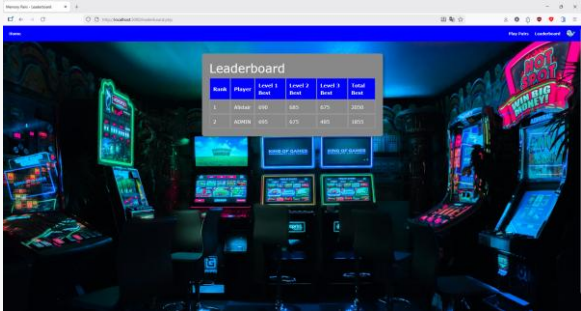
- Leaderboard file writes are not currently protected by an exclusive lock (flock). Under simultaneous submission from two users, a read-modify-write race condition could cause one submission to be lost. Since the website isn't being deployed on a large scale, I doubt this will be an issue needing of an urgent fix.
- The attempt counter increments only after a full group selection of matchSize cards is made. This is the intended definition, but it means the first card click in a group does not count as an attempt, which may feel unintuitive.
- The avatar image assets (avatar1.png–avatar6.png) must be present in the project directory on the VM; there is no fallback if an asset is missing.
- Cookie expiry is 7 days. If a user clears cookies, the profile is lost and re-registration is required, as there is no server-side account store.

Part 2: Development Log

The table below records each major development session in chronological order. Each entry includes the date, start time, duration, and a description of the work completed. Screenshot placeholders indicate where captured images from VS Code and the browser should be inserted - at least one per session, covering six distinct development stages as required.

Date	Start	Duration	Work Description
05 Feb 2026	14:00	2.5 hrs	Session 1 - Environment Setup and Project Scaffolding. Provisioned the LAMP stack on the Azure VM: installed Apache2 and PHP 8.1, confirmed the server was reachable via the public IP, and created the /var/www/html/ project directory. Wrote the four required PHP files as empty shells (index.php, registration.php, pairs.php, leaderboard.php) and the global stylesheet (styles.css). Verified Apache was serving PHP by loading a phpinfo() test page.
			
12 Feb 2026	10:30	3 hrs	Session 2 - Navbar, Shared Styling and Landing Page. Implemented the navbar markup in each page file. Applied Verdana 12px bold, blue background, and flexbox alignment (Home left-aligned; Play Pairs and Leaderboard/Register right-aligned). Wrote the isset(\$_COOKIE['username']) conditional in each file to toggle Leaderboard vs Register. Completed index.php: welcome message with cookie-based username, 'Click here to play' button, and the unregistered fallback paragraph. Applied the arcade background image to #main with background-size: cover and background-position: center.
			
19 Feb 2026	11:00	3.5 hrs	Session 3 - Registration Form, Validation and Avatar Selector. Built registration.php: form posts to itself; PHP preg_match validates the username against the specified invalid character regex; error message renders beneath the input with the prior value repopulated. On success, setcookie() writes username and avatar with a 7-day expiry. Built the avatar grid: six avatar image assets (avatar1.png–avatar6.png) as radio inputs inside labels, with JavaScript adding a selected highlight class on change.

Date	Start	Duration	Work Description
			
26 Feb 2026	13:00	4 hrs	Session 4 - Game Board Layout and Card Flip Animation. Built the game container in pairs.php: grey background div with 5px box shadow, CSS grid for card layout. Each card is a div with two child faces (.card-front and .card-back) using CSS 3D transforms (rotateY(180deg) on the .flipped class). Implemented the emoji pool array and the sampling/duplication/shuffle (Fisher-Yates) logic that populates a level's card set. Wired the click handler to add the .flipped class and track selected cards. 
05 Mar 2026	10:00	5 hrs	Session 5 - Full Game Logic, Level Progression and Scoring. Implemented the complete game state machine: level start (shuffle and render), selection accumulation up to matchSize, attempt increment on full group selection, match evaluation, lock on success, flip-back on failure (600 ms delay). Built level definitions: Level 1 pairs (matchSize 2), Level 2 triples (matchSize 3), Level 3+ quads (matchSize 4) with increasing card counts. Implemented the scoring algorithm: base score minus failed-attempt penalty, plus elapsed-time bonus. Wired the gold background trigger when live level score exceeds stored level best. 
12 Mar 2026	14:00	3.5 hrs	Session 6 - Leaderboard, Score Submission and Final Testing. Implemented leaderboard.php: POST handler reads leaderboard_store.json, updates \$store['users'][\$username] with new per-level and total bests, writes the file back. GET handler sorts and renders the blue-header table with border-spacing: 2px and per-level score columns. Wired the end-of-game panel in pairs.php: Submit triggers fetch() POST; Play Again resets all state to zero and

Date	Start	Duration	Work Description
			restarts. Ran cross-browser tests in Chrome, Firefox and Edge. Verified multi-user persistence by submitting from two separate private browser windows and confirming both records appeared on the leaderboard.
			

Total logged development time: approximately 21.5 hours across six sessions.

References

Carl Raw (2019). Arcade photograph. Unsplash. Available at: <https://unsplash.com/> [Accessed February 2026].

Bootstrap (2024). Bootstrap 5 Documentation. Available at: <https://getbootstrap.com/docs/5.0/> [Accessed February 2026].

Mozilla Developer Network (2024). CSS transform property. Available at: <https://developer.mozilla.org/en-US/docs/Web/CSS/transform> [Accessed March 2026].

Mozilla Developer Network (2024). Fisher-Yates shuffle. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random [Accessed March 2026].

PHP Manual (2024). setcookie. Available at: <https://www.php.net/manual/en/function.setcookie.php> [Accessed March 2026].

Flaticon.com. (2026). Available at: <https://www.flaticon.com/search?word=animal> [Accessed March. 2026].